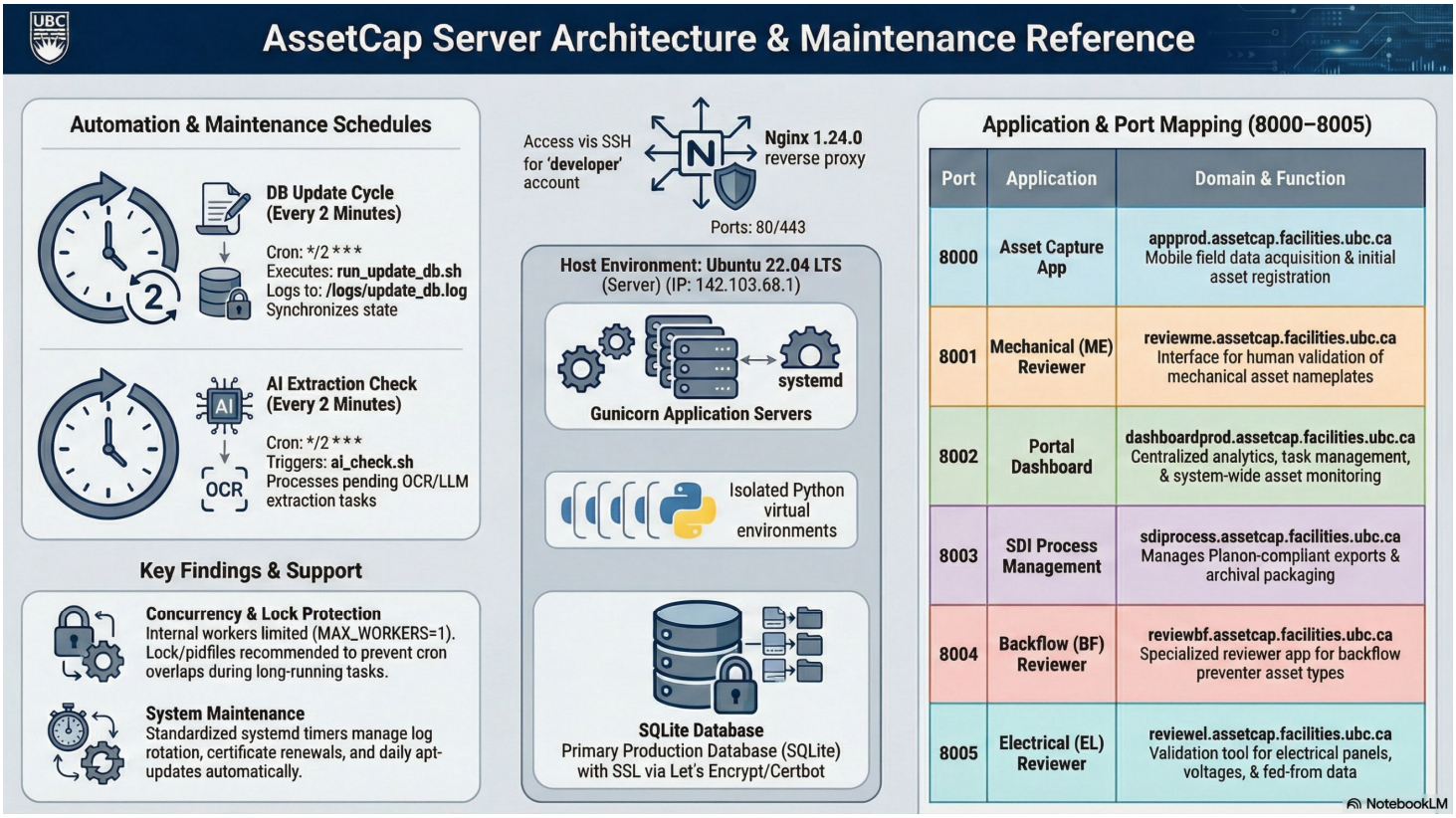
AssetCap Server Setup and Maintenance — Table of Contents

Quick Introduction

The AssetCap Server Setup and Maintenance Manual (v2.2) is an operational guide for administering the centralized Ubuntu 22.04 LTS server that hosts the UBC Asset Capture platform. It documents the server's core stack/Nginx as the reverse proxy, Gunicorn managed by systemd (one service per Flask app), and isolated Python virtual environments (venv) per application, along with the current production database usage (SQLite, where applicable). The manual also serves as an inventory and routing reference, mapping each web application (Capture App, Dashboard, Review apps, and SDI Process) to its production domain and local Gunicorn port (8000–8005). In addition, it outlines key operational workflows for stability and support, including post-deployment checks, logging and troubleshooting commands, and scheduled automation (developer cron jobs running every two minutes for database updates and AI checks).



1. Server Environment & Network Topology

1.1 Host Specifications

- **IP Address:** 142.103.68.1
- **Operating System:** Ubuntu 22.04 LTS
- **Default User:** developer
- **SSH Access:** ssh developer@142.103.68.1 (Sudo privileges enabled; requires password).

1.2 Core Technology Stack

Component	Implementation	Details
Reverse Proxy	Nginx 1.24.0	Terminating TLS 1.3 and handling HTTP/2 for dashboard.
App Server	Gunicorn	Python WSGI HTTP Server managed by systemd.
Databases	SQLite	File-based persistence using WAL mode.
Python Env	Isolated venvs	Python 3.10+ with application-specific virtual environments.

System Dependency Note: The system requires libgl1 to be installed at the OS level to support OpenCV image processing within the extraction and review modules.

1.3 Global Directory Structure

The application ecosystem resides entirely within the developer home directory:

- /home/developer/asset_capture_app_dev: Core Capture App, shared data, and master databases.
- /home/developer/review/: Host for Review App subdirectories (ME, BF, EL).
- /home/developer/Dashboard: Centralized management portal and analytics.
- /home/developer/auth_service: Global authentication persistence and user management scripts.
- /home/developer/SDI_process: Planon-compliant export and archival engine.

2. Domain & Application Inventory

2.1 Platform Handoff Flow

The platform operates as a linear pipeline ensuring data integrity from field to archive:

1. **Capture App:** Technicians scan QR codes and upload field photos.
2. **Extraction API:** OCR and GPT-4o vision models generate structured JSON data.
3. **Review Apps:** Engineering validation through a "Human-in-the-Loop" UI for approval or overrides.
4. **SDI Process:** Approved assets are packaged, validated for Planon compliance, and archived.

2.2 Service Mapping

Domain	Port	Systemd Service	File Path
appprod.assetcap.facilities.ubc.ca	8000	assetcap-app.service	/home/developer/asset_capture_app_dev
reviewme.assetcap.facilities.ubc.ca	8001	assetcap-reviewme.service	/home/developer/review/Asset_dasboard_browser_ME
dashboardprod.assetcap.facilities.ubc.ca	8002	assetcap-dashboard.service	/home/developer/Dashboard
sdiprocess.assetcap.facilities.ubc.ca	8003	sdi_process.service	/home/developer/SDI_process
reviewbf.assetcap.facilities.ubc.ca	8004	assetcap-bf.service	/home/developer/review/Asset_dasboard_browser_BF
reviewel.assetcap.facilities.ubc.ca	8005	assetcap-el.service	/home/developer/review/Asset_dashboard_browser_EL

2.3 Module Responsibilities

- **Capture App:** Mobile-optimized interface for 10-digit QR scanning and standardized image sequencing.
- **Extraction API:** Hybrid OCR/LLM processing utilizing Tesseract and GPT-4o.
- **Review Apps (ME, BF, EL):** Asset-specific dashboards for New (Process 0), Update (Process 1), and Manual Entry (Process 2) states.
- **Dashboard:** SPA (Single Page Application) for cross-service analytics and task triggering.
- **SDI Process:** Final validation script (asset_template_validation_ver00.py) and Excel packaging.

3. Nginx Configuration & Security

3.1 Reverse Proxy Layout

- **Configuration Directory:** /etc/nginx/sites-available/
- **Deployment Directory:** /etc/nginx/sites-enabled/ (symlinked)
- **Verification:** Always run sudo nginx -t before reloading.

3.2 Vhost & TLS Certificate Mapping

Domain	Port	Enabled Link	TLS Cert Path (Let's Encrypt)
appprod	8000	assetcap-app	/etc/letsencrypt/live/appprod.assetcap.facilities.ubc.ca/
reviewel	8005	assetcap-el	/etc/letsencrypt/live/reviewel.assetcap.facilities.ubc.ca/
reviewme	8001	assetcap-reviewme	/etc/letsencrypt/live/reviewme.assetcap.facilities.ubc.ca/
dashboardprod	8002	dashboardprod	/etc/letsencrypt/live/dashboardprod.assetcap.facilities.ubc.ca/*
reviewbf	8004	reviewbf	/etc/letsencrypt/live/reviewbf.assetcap.facilities.ubc.ca/
sdiprocess	8003	sdi_process	/etc/letsencrypt/live/sdiprocess.assetcap.facilities.ubc.ca/

**Note: dashboardprod is specifically configured for* `listen 443 ssl http2` *to optimize asset delivery.*

3.3 Security Guardrails

1. **HTTPS Enforcement:** Mandatory TLS 1.2+ for all production traffic.
2. **SQL Safety:** All queries utilize? parameterized placeholders to mitigate injection risks.
3. **No-Secrets Policy:** Credentials and API keys must be stored in local .env files, never in source control.

4. **Safe Dictionary Parsing:** Use ast.literal_eval() for evaluating Python dictionaries from the asset dictionary files to prevent RCE.
5. **Hallucination Defense:** Automated regex validation (e.g., UBC Tag patterns) is applied to AI outputs to filter erroneous LLM-generated data.

4. Application Management & Systemd Services

4.1 Service Control Commands

- **Reload Unit Files:** sudo systemctl daemon-reload
- **Restart Service:** sudo systemctl restart <service-name>
- **Check Health:** sudo systemctl status <service-name>

4.2 Gunicorn Targets

Multiple services utilize shared logic but operate out of isolated directories:

- **Capture App:** asset_plate_reviewer:app (Path: .../asset_capture_app_dev)
- **ME Review:** asset_plate_reviewer:app (Path: .../review/Asset_dasboard_browser_ME)
- **BF Review:** asset_plate_reviewer_bf:app
- **EL Review:** Asset_dashboard_EL:app
- **Dashboard:** Asset_portal_dashboard:app
- **SDI Process:** app:app

4.3 Log Inspection

Access service logs via journald:

- **Real-time Stream:** sudo journalctl -u <service-name> -f
- **Recent History:** sudo journalctl -u <service-name> -n 100 --no-pager

5. Database Architecture & Persistence

5.1 Database Inventory

Database	Ownership	Scope
User_control.db	Auth Service	User roles, bcrypt-hashed passwords.
QR_codes.db	Shared	19 Operational tables (tracking, AI status, reviews).

5.2 Core Schema Reference

The QR_codes.db is the primary operational store, containing 19 tables. Key tables include:

- **QR_codes**: The master registry for tags, building associations, and ai_status.
- **QR_code_assets**: Audit log for user interactions and field capture events.

- **sdi_dataset / sdi_dataset_EL**: Staging for approved assets before packaging.
- **json_files**: Physical file-to-DB mapping required for atomic rename operations.

5.3 SQLite Maintenance

The server uses Write-Ahead Logging (WAL). To maintain integrity and prevent WAL file bloating, execute the checkpoint script regularly: bash /home/developer/asset_capture_app_dev/data/checkpoint_db.sh

6. Automated Workflows & Scheduled Tasks

6.1 Cron Job Schedule & Locking

The developer user manages automation via crontab. Note the use of flock as a defense-in-depth measure to prevent execution overlap:

DB Update: Synchronize captures and status every 2 minutes

*/2 * * * * flock -n /tmp/update_db.lock /home/developer/run_update_db.sh >> /home/developer/logs/update_db.log 2>&1

AI Check: Trigger hybrid extraction for pending assets every 2 minutes

*/2 * * * * flock -n /tmp/ai_check.lock /home/developer/ai_check.sh

6.2 Concurrency & Internal Workers

The Extraction API manages internal concurrency via ThreadPoolExecutor. Settings are typically locked to MAX_WORKERS=1 or MAX_WORKERS=2 to ensure thread safety when performing I/O-intensive OCR tasks.

6.3 Extraction Flow & Completeness Guard

To protect data integrity, the system utilizes the **Completeness Guard** heuristic:

1. **Extraction:** Tesseract OCR and LLM generate structured data.
2. **Scoring:** A score is calculated: (populated required fields / total required fields) * 100.
3. **Overwrite Logic:** Data is only saved if New Score >= Existing Score.
4. **Loop Prevention:** The system sets ai_status=1 **regardless** of whether the score was high enough to overwrite the file. This prevents the cron job from repeatedly attempting to process low-quality images.

7. File Tracking & Logic Engines

7.1 Image & JSON Naming Conventions

- **Images:** <QR> <Building> <TYPE> - <SEQ>.jpg (e.g., 0000183710 100 EL-1.jpg)
- **JSON:** <QR>_<TYPE>_<Building>.json

- **Validation Regex:** ^([T]?\d+)\s+(\d+(?:-\d+)?)\s+([A-Z]{2})\s*-\s*([0-3])$

7.2 Atomic Parameter Updates

Updates to core parameters (QR code renames or building changes) are handled by the parameter_update_service.py engine. This engine ensures consistency across the 19 tables in QR_codes.db and the physical file system:

1. **Snapshot:** Backup target files to rollback memory.
2. **OS Rename:** Atomically rename physical .jpg and .json files.
3. **DB Cascade:** Execute string updates across 8+ key operational tables (QR_codes, sdi_dataset, json_files, etc.).
4. **JSON Patch:** Update internal JSON keys and set modified=True.
5. **Commit:** Finalize transaction. Any failure triggers a full rollback of all five steps.

7.3 Field Normalization & Limits

All fields are sanitized and normalized via validators_shared.py:

- **General Limits:** Serial Numbers are capped at **64 characters**; Models are capped at **80 characters**.
- **Mechanical (ME):** Manufacturer brand matching and title-casing.
- **Electrical (EL):** UBC Asset Tags **must** utilize the PNL- prefix (e.g., PNL-123).
- **Backflow (BF):** Diameter normalization (fractional inches, ensuring " suffix).

8. Deployment & Routine Maintenance

8.1 Deployment Workflow

For manual updates to any application:

1. Navigate to application path: cd /home/developer/<app_dir>
2. Fetch source: git pull origin main
3. Enter environment: source venv/bin/activate
4. Update requirements: pip install -r requirements.txt
5. Restart service: sudo systemctl restart <service-name>

8.2 Post-Deployment Validation

Perform the following checks to verify system stability:

- **Process Check:** sudo systemctl status <service-name>
- **Networking:** sudo ss -ltnp | grep :<port>
- **Log Check:** sudo journalctl -u <service-name> -n 50

8.3 Maintenance Tasks

- **Disk Monitoring:** df -h (Monitor /home for image storage growth).
- **Cleanup:** pip cache purge to remove unnecessary installation artifacts.
- **SSL Maintenance:** sudo certbot renew --dry-run to verify Let's Encrypt automation.
- **WAL Checkpoint:** Execute the manual database checkpoint script weekly.